

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №10
по дисциплине «Вычислительная математика»
Тема: Интерполяция Ньютона. Интерполяция сплайнами.

Студентка гр. 4342

Волкова В.Н.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Изучить методы интерполяции функций с использованием полинома Ньютона и кубических сплайнов, реализовать их программно, а также провести сравнительный анализ точности и устойчивости получаемых приближений на заданной функции.

Основные теоретические положения.

Интерполяция функций применяется для приближённого восстановления функции по её значениям в конечном числе точек. Пусть задана функция $f(x)$ и набор узлов интерполяции $x_0, x_1, \dots, x_n$. Требуется построить функцию $P(x)$, удовлетворяющую условиям:

$$P(x_i) = f(x_i), \quad i = 0, 1, \dots, n.$$

В данной работе рассматриваются два метода интерполяции: полином Ньютона и кубические сплайны.

Интерполяционный полином Ньютона строится в виде:

$$P(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + \dots + c_n(x - x_0) \dots (x - x_{n-1}),$$

где коэффициенты c_i вычисляются с помощью разделённых разностей. Такая форма удобна тем, что позволяет последовательно добавлять новые узлы без пересчёта всего полинома.

Однако при увеличении числа узлов степень полинома возрастает, что может приводить к численной неустойчивости и появлению значительных колебаний на концах интервала. Это ограничивает применение полиномов высокой степени на равномерных сетках.

Альтернативой является использование кубических сплайнов. Кубический сплайн представляет собой функцию, состоящую из набора полиномов третьей степени, заданных на каждом промежутке $[x_i, x_{i+1}]$:

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3.$$

При построении сплайна накладываются условия:

- совпадение значений функции в узлах интерполяции;
- непрерывность функции на всём интервале;
- непрерывность первой и второй производных;
- дополнительные граничные условия (в данной работе используется натуральный сплайн, для которого вторая производная на концах равна нулю).

В отличие от глобального интерполяционного полинома, сплайн является кусочной функцией, что обеспечивает более высокую устойчивость и отсутствие сильных колебаний. Благодаря этому кубические сплайны широко применяются на практике для аппроксимации функций и обработки экспериментальных данных.

Таким образом, оба метода позволяют построить интерполяцию функции, однако их свойства существенно различаются: полином Ньютона удобен теоретически, а кубические сплайны обеспечивают более стабильное и качественное приближение.

Выполнение работы.

В качестве исследуемой функции выбрана функция:

$$f(x)=\arccos(x^2)-x.$$

Интерполяция выполняется на отрезке: $[-1,1]$.

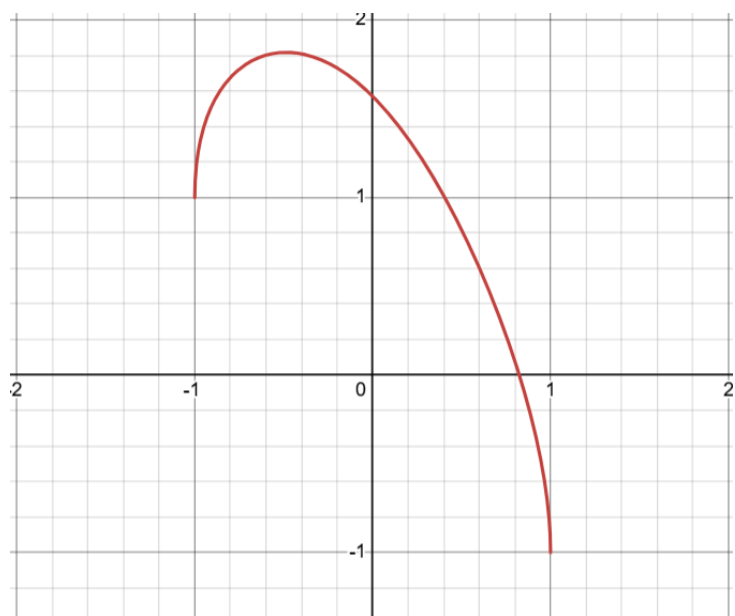


Рисунок 1 — график функции $f(x)$

Функция является непрерывной и гладкой на данном интервале, что позволяет применять как полиномиальную интерполяцию, так и интерполяцию с помощью кубических сплайнов.

Для построения интерполяции отрезок $[-1, 1]$ разбивается на заданное число узлов n , равномерно распределённых по интервалу. В каждом узле вычисляются значения функции $f(x_i)$, которые используются для построения интерполяционных функций.

В работе реализованы два метода:

1. Полином Ньютона

На основе полученных узлов вычисляются разделённые разности, после чего строится интерполяционный полином Ньютона. Значение полинома вычисляется в произвольных точках интервала.

2. Кубический сплайн

Для тех же узлов строится натуральный кубический сплайн, представляющий собой совокупность полиномов третьей степени на каждом подынтервале. Сплайн обеспечивает непрерывность функции и её первых двух производных.

Для анализа результатов на одном графике строятся:

- исходная функция $f(x)$;
- интерполяционный полином Ньютона;
- кубический сплайн;
- узлы интерполяции.

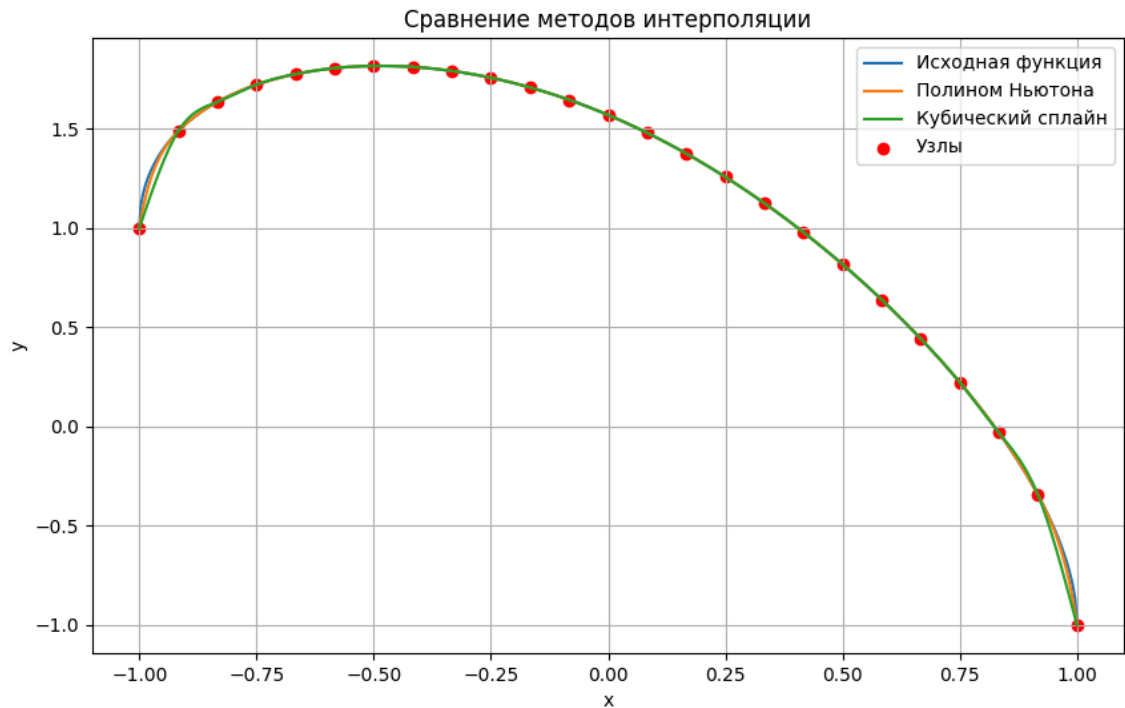


Рисунок 2 — график функции и её интерполяций (кол-во узлов: 25)

На графике можно визуально сравнить точность приближений. Полином Ньютона точно проходит через узлы интерполяции, однако при большом числе узлов может наблюдаться ухудшение поведения на концах интервала. Кубический сплайн также проходит через узлы, но при этом сохраняет гладкость и более устойчивое поведение на всём интервале.

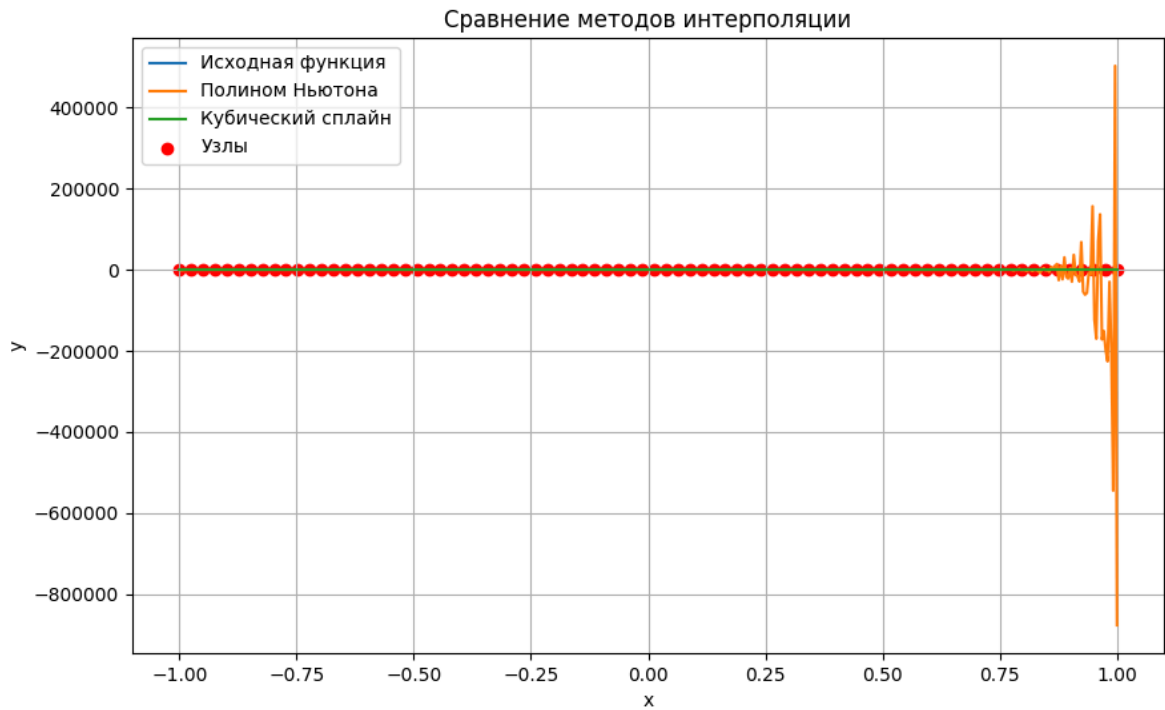


Рисунок 3 — график функции и её интерполяций (кол-во узлов: 80)

Дополнительно проводилось исследование влияния числа узлов интерполяции на точность приближения. При увеличении числа узлов точность интерполяции возрастает, однако для полинома Ньютона это сопровождается ростом колебаний, тогда как сплайн остаётся устойчивым.

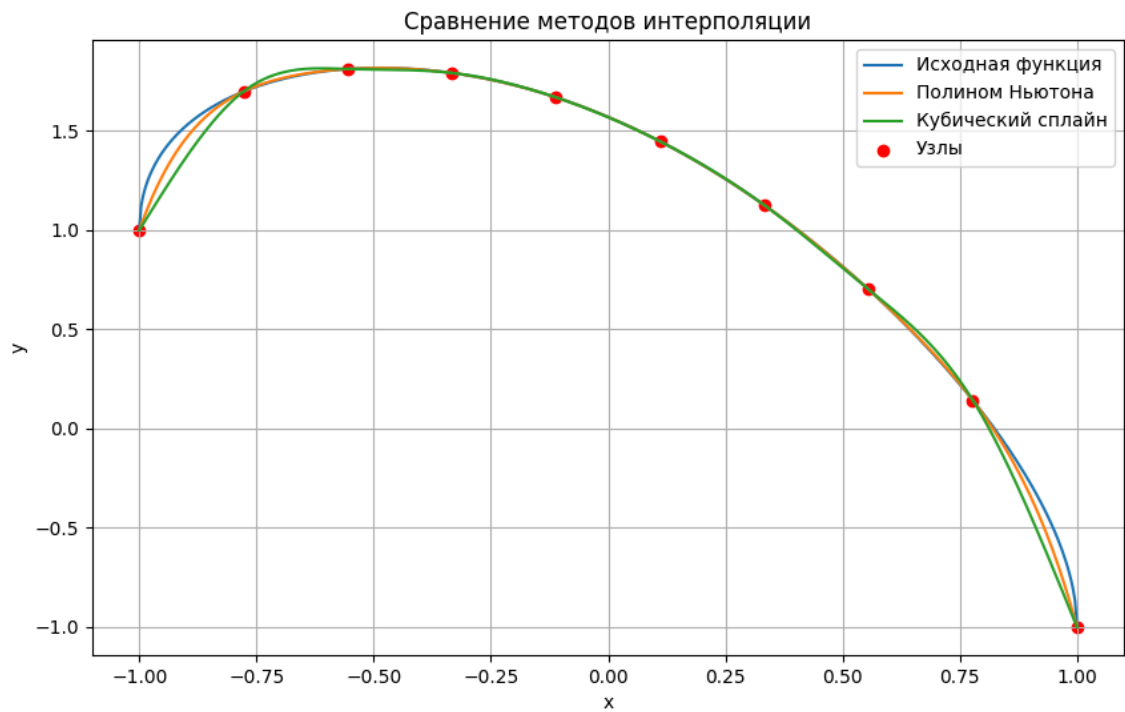


Рисунок 4 — график функции и её интерполяций (кол-во узлов: 10)

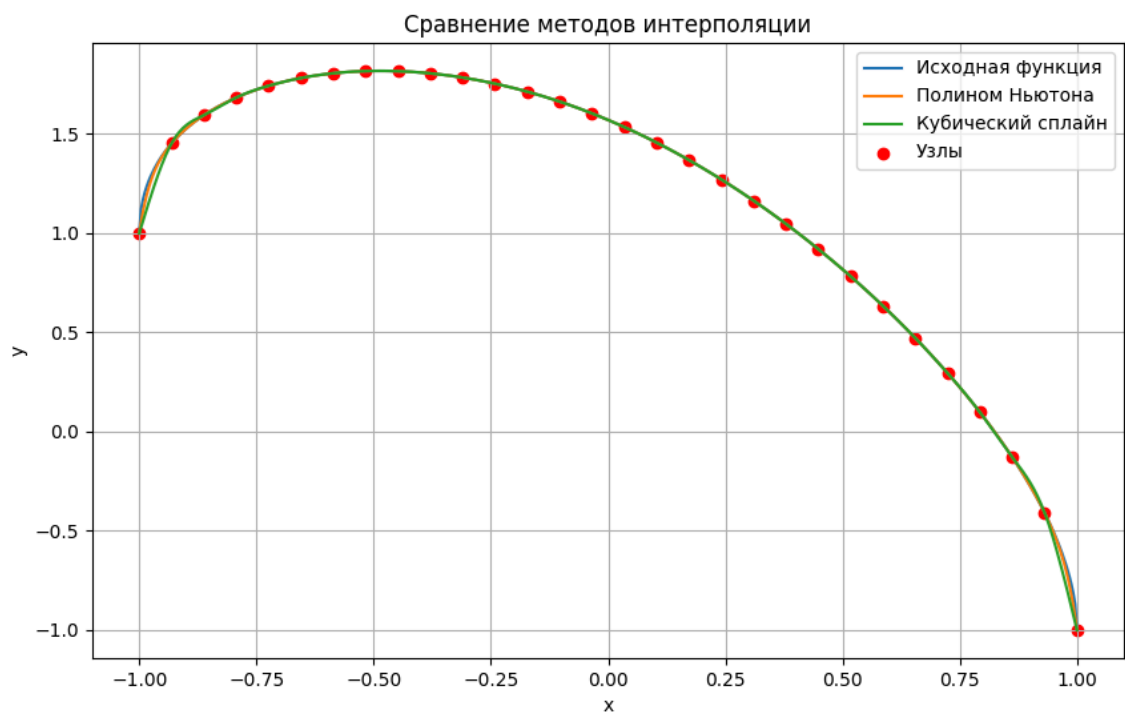


Рисунок 5 — график функции и её интерполяций (кол-во узлов: 30)

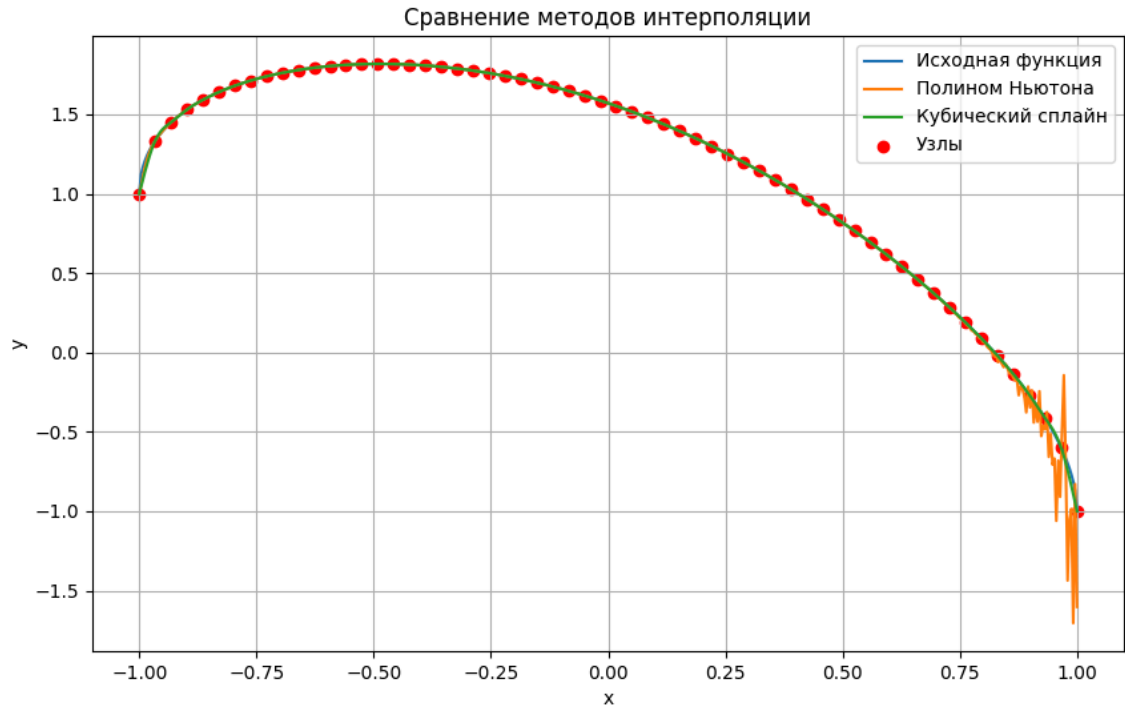


Рисунок 6 — график функции и её интерполяций (кол-во узлов: 60)

Выводы.

В ходе лабораторной работы были реализованы методы интерполяции функций: полином Ньютона и кубические сплайны. Для заданной функции $f(x)=\arccos(x^2)-x$ на отрезке $[-1,1]$ построены интерполяционные приближения и проведено их сравнение.

Установлено, что оба метода точно воспроизводят значения функции в узлах интерполяции. Однако их поведение между узлами существенно различается. Полином Ньютона при увеличении числа узлов начинает проявлять численную неустойчивость и может давать значительные колебания на концах интервала. Это связано с высокой степенью интерполяционного полинома.

Кубический сплайн, напротив, демонстрирует устойчивое и гладкое поведение на всём интервале. Благодаря своей кусочно-полиномиальной структуре он не подвержен сильным колебаниям и обеспечивает более качественное приближение функции между узлами.

Также было установлено, что увеличение числа узлов приводит к повышению точности интерполяции для обоих методов, однако для полинома Ньютона это сопровождается ухудшением устойчивости, тогда как сплайн сохраняет стабильность.

Таким образом, для практического применения, особенно при большом числе узлов, предпочтительнее использовать кубические сплайны, поскольку они обеспечивают более надёжное и гладкое приближение функции.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл `interpolation.py`

```
import math
import argparse
import matplotlib.pyplot as plt

def f(x):
    return math.acos(x * x) - x

def make_nodes(a, b, nodes):
    x = []
    y = []

    for i in range(nodes):
        xi = a + i * (b - a) / (nodes - 1)
        x.append(xi)
        y.append(f(xi))

    return x, y

def get_limited_indices(count, edge_count=5):
    if count <= edge_count * 2 + 2:
        return list(range(count))

    indices = []

    for i in range(edge_count):
        indices.append(i)

    indices.append(None)

    for i in range(count - edge_count, count):
        indices.append(i)

    return indices

def divided_differences(x, y):
    n = len(x)
    coef = y[:]

    for j in range(1, n):
        for i in range(n - 1, j - 1, -1):
            coef[i] = (coef[i] - coef[i - 1]) / (x[i] - x[i - j])

    return coef

def newton_value(x_nodes, coef, x):
    result = coef[-1]

    for i in range(len(coef) - 2, -1, -1):
        result = result * (x - x_nodes[i]) + coef[i]
```

```

    return result

def newton_to_power(x_nodes, coef):
    poly = [coef[-1]]

    for k in range(len(coef) - 2, -1, -1):
        new_poly = [0.0] * (len(poly) + 1)

        for i in range(len(poly)):
            new_poly[i] += -x_nodes[k] * poly[i]
            new_poly[i + 1] += poly[i]

        new_poly[0] += coef[k]
        poly = new_poly

    return poly

def build_natural_cubic_spline(x, y):
    n = len(x) - 1
    h = [x[i + 1] - x[i] for i in range(n)]

    alpha = [0.0] * (n + 1)

    for i in range(1, n):
        alpha[i] = (
            3 / h[i] * (y[i + 1] - y[i])
            - 3 / h[i - 1] * (y[i] - y[i - 1])
        )

    l = [0.0] * (n + 1)
    mu = [0.0] * (n + 1)
    z = [0.0] * (n + 1)

    l[0] = 1.0

    for i in range(1, n):
        l[i] = 2 * (x[i + 1] - x[i - 1]) - h[i - 1] * mu[i - 1]
        mu[i] = h[i] / l[i]
        z[i] = (alpha[i] - h[i - 1] * z[i - 1]) / l[i]

    l[n] = 1.0

    a_coef = y[:-1]
    b_coef = [0.0] * n
    c_coef = [0.0] * (n + 1)
    d_coef = [0.0] * n

    for j in range(n - 1, -1, -1):
        c_coef[j] = z[j] - mu[j] * c_coef[j + 1]
        b_coef[j] = (
            (y[j + 1] - y[j]) / h[j]
            - h[j] * (c_coef[j + 1] + 2 * c_coef[j]) / 3
        )
        d_coef[j] = (c_coef[j + 1] - c_coef[j]) / (3 * h[j])

    return a_coef, b_coef, c_coef[:-1], d_coef

def spline_value(x_nodes, spline, x):
    a_coef, b_coef, c_coef, d_coef = spline

```

```

interval_index = len(x_nodes) - 2

for i in range(len(x_nodes) - 1):
    if x_nodes[i] <= x <= x_nodes[i + 1]:
        interval_index = i
        break

dx = x - x_nodes[interval_index]

return (
    a_coef[interval_index]
    + b_coef[interval_index] * dx
    + c_coef[interval_index] * dx ** 2
    + d_coef[interval_index] * dx ** 3
)

def print_nodes(x_nodes, y_nodes):
    print("Узлы интерполяции:")
    print("-" * 36)
    print(f"{'i':>3} | {'x_i':>12} | {'f(x_i)':>12}")
    print("-" * 36)

    for i in get_limited_indices(len(x_nodes)):
        if i is None:
            print(f"{'...':>3} | {'...':>12} | {'...':>12}")
        else:
            print(f"{'i':>3} | {x_nodes[i]:>12.6f} | {y_nodes[i]:>12.6f}")

    print("-" * 36)
    print()

def print_newton_coefficients(coef):
    print("Коэффициенты многочлена Ньютона:")
    print("-" * 36)
    print(f"{'i':>3} | {'c_i':>20}")
    print("-" * 36)

    for i in get_limited_indices(len(coef)):
        if i is None:
            print(f"{'...':>3} | {'...':>20}")
        else:
            print(f"{'i':>3} | {coef[i]:>20.10e}")

    print("-" * 36)
    print()

def print_power_coefficients(poly):
    print("Коэффициенты в степенной форме:")
    print("-" * 36)
    print(f"{'i':>3} | {'p_i':>20}")
    print("-" * 36)

    for i in get_limited_indices(len(poly)):
        if i is None:
            print(f"{'...':>3} | {'...':>20}")
        else:
            print(f"{'i':>3} | {poly[i]:>20.10e}")

    print("-" * 36)

```

```

print()

def print_spline_coefficients(x_nodes, spline):
    a_coef, b_coef, c_coef, d_coef = spline

    print("Коэффициенты кубического сплайна:")
    print("-" * 100)
    print(
        f"{'i':>3} | {'отрезок':>17} | "
        f"{'a_i':>14} | {'b_i':>14} | {'c_i':>14} | {'d_i':>14}"
    )
    print("-" * 100)

    for i in get_limited_indices(len(a_coef)):
        if i is None:
            print(
                f"{'...':>3} | {'...':>17} | "
                f"{'...':>14} | {'...':>14} | {'...':>14} | {'...':>14}"
            )
        else:
            interval = f"[{x_nodes[i]:.4f}; {x_nodes[i + 1]:.4f}]"

            print(
                f"{'i':>3} | {'interval':>17} | "
                f"{'a_coef[i]':>14.8f} | "
                f"{'b_coef[i]':>14.8f} | "
                f"{'c_coef[i]':>14.8f} | "
                f"{'d_coef[i]':>14.8f}"
            )

    print("-" * 100)
    print()

def show_plot(a, b, x_nodes, y_nodes, newton_coef, spline):
    xs = []
    original_values = []
    newton_values = []
    spline_values = []

    point_count = 500

    for i in range(point_count):
        x = a + i * (b - a) / (point_count - 1)

        xs.append(x)
        original_values.append(f(x))
        newton_values.append(newton_value(x_nodes, newton_coef, x))
        spline_values.append(spline_value(x_nodes, spline, x))

    plt.figure(figsize=(10, 6))

    plt.plot(xs, original_values, label="Исходная функция")
    plt.plot(xs, newton_values, label="Полином Ньютона")
    plt.plot(xs, spline_values, label="Кубический сплайн")

    plt.scatter(x_nodes, y_nodes, label="Узлы", color="red")

    plt.title("Сравнение методов интерполяции")
    plt.xlabel("x")
    plt.ylabel("y")
    plt.grid(True)

```

```

plt.legend()

plt.show()

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--nodes", type=int, default=9)

    args = parser.parse_args()

    a = -1
    b = 1
    nodes = args.nodes

    if nodes < 2:
        raise ValueError("Количество узлов должно быть не меньше 2.")

    x_nodes, y_nodes = make_nodes(a, b, nodes)

    newton_coef = divided_differences(x_nodes, y_nodes)
    #power_coef = newton_to_power(x_nodes, newton_coef)
    spline = build_natural_cubic_spline(x_nodes, y_nodes)

    show_plot(a, b, x_nodes, y_nodes, newton_coef, spline)

if __name__ == "__main__":
    main()

```